Asistente de recomendación de películas mediante recuperación de información y procesamiento de lenguaje natural

Danna Andrea Rodriguez Galarcio
Ingeniería de Sistemas, Facultad de Ingeniería
Universidad de San Buenaventura
Cali, Colombia
darodriguezg4@correo.usbcali.edu.co

***Abstract*—**
***Index Terms*—clasificación de texto, TF-IDF, modelos de lenguaje, embeddings multilingües, atención al cliente, CRISP-ML(Q)**

I. Introducción

En los ultimos años se ha visto un aumento en el acceso a peliculas a traves de plataformas de streaming, estas plataformas cuentan con un gran numero de usuarios los cuales en muchas ocasiones tienen la necesidad de poder contar con una herramienta que les permita encontrar contenido que se ajuste a sus intereses.

En este proyecto se propone el desarollo de un asistente virtual para la recomendacion de peliculas el cual permitira a los usuarios realizar sus preguntas sobre las peliculas del catalogo y recibir informacion sobre las mismas, teniendo en cuenta caracteristias como titulo, genero, sinopsis, entre otros.

Es en este contexto que el poder implementar un sistema de clasificacion automatica de texto resulta realmente util, el procesamiento PLN (Procesamiento del Lenguaje Natural) permite desarrollar sistemas capaces de procesar y analizar textos dados por el usuario y relacionarlos directamente con la informacion disponible, en este caso, informacion sobre peliculas.

Existen diferentes tecnicas que pueden considerarse para poder implementar un sistema con esas caracteristicas, en este caso, el sistema propone el uso de dos enfoques:

El primer enfoque consiste en la implementacion de TF-IDF (Term Frequency-Inverse Document Frequency) y similitud del coseno, el cual permite poder medir cual es la similutd del texto dado por el usuario en comparacion con la informacion disponible en el catalogo de peliculas,para de esta manera poder determinar cual es la pelicula que mas se ajusta a la pregunta realizada por el usuario mediante la coincidencia de las palabras.

El segundo enfoque consiste en usar un modelo preentrenado de embeddings, este modelo permite poder representar el texto dado por el usuario y la informacion disponible en el catalogo de peliculas en un espacio vectorial, este metodo tambien usa la similitud del coseno para pode medir la similitud entre los textos y de esta manera, poder identificar las peliculas que mas se relacionen a lo que pidio el usuario. Adicionalmente, cuenta con la incorporacion de Qween, un modelo de lenguaje LLM (Large Language Model) el cual permite poder generar embeddings de manera mas eficiente y precisa, este se incluyo ya que permite poder redactar una respuesta mas completa apartir de las peliculas encontradas.

La evaluacion se centrara en la comparacion entre estos dos enfoques para asi poder determinar la capacidad de cada enfoque para poder recuperar peliculas relevantes a partir de la pregunta realizada por el usuario, ademas de poder determinar cual de los dos enfoques es mas eficiente en cuanto a tiempo de respuesta y consumo de recursos. Las metricas de evaluacion implementadas consisten en Top-1, Recall@5, rango reciproco medio (MRR) y tiempo de respuesta.

A. Objetivos

Desarrollar un asistente conversacional que permita a los usuarios realizar preguntas y recibir respuestas en espaÃ±ol para la recomendacion de peliculas mediante la implementacion y comparacion de dos metodos de recuperacion de informacion, uno basado en TF-IDF y otro con embeddings multilingues, ambos con similitud del coseno.

Los objetivos especificos son:

1) Preparar un conjunto de datos de peliculas que incluya informaciión textual relevante, como titulos, generos y sinopsis, para utilizarlo en la recuperacion de contenidos a partir de consultas en español.
2) Implementar un metodo de recuperacion basado en TF-IDF y similitud del coseno que permita comparar las consultas de los usuarios con la informacion textual de las peliculas y recuperar los resultados mas relacionados.
3) Implementar un metodo de recuperacion semantica mediante embeddings multilingues y similitud del coseno, con el fin de identificar peliculas relacionadas con la intencion de las consultas, incluso cuando existan diferencias entre los terminos utilizados por el usuario y los textos del catalogo.
4) Evaluar y comparar los dos metodos de recuperacion mediante las metricas Top-1, Recall@5, rango reciproco

medio (MRR) y tiempo de respuesta, utilizando un conjunto de consultas de prueba y criterios de evaluacion definidos previamente.

B. Justificación

Para los usuarios, el realizar una busqueda de peliculas puede resultar una tarea tediosa e incluso compleja cuando los usuarios no conocen el titulo de la pelicula o en general, no saben que pelicula quieren ver y simplemente necesitan recomendaciones de un genero en especifico. Para este tipo de consultas, es importante considerar tecnicas de procesamiento de lenguaje natural que permiten poder relacionar las necesidades expresadas en las preguntas del usuario con las caracteristicas de las peliculas en el catalogo.

Este proyecto se justifica porque permite comparar dos enfoques distintos para poder cumplir con el proposito del proyecto, TF-IDF y embeddings multilingues, ambos combinados con la similitud del coseno. Esta combinacion permite determinar si las diferencias en los modelos se reflejan en el desempeño del sistema. La eleccion de las tecnicas tambien se justifica ya que permite construir un prototipo funcional con herramientas de acceso libre y gratuito sin depender de servicios que requieran de una suscripcion o pago.

Por otro lado, la incorporacion de un LLM permite transformar los resultados obtenidos en una respuesta mas completa y coherente para el usuario, esta capa se enfoca en la facilitacion de la presentacion de la informacion en pantalla, su funcionalidad se centra en unicamente redactar las respuestas a partir de la informacion obtenida en la busqueda, para de esta manera evitar el riesgo de que el modelo genere afirmaciones que no se encuentren respaldadas.

En general, el valor academico de este proyecto radica en que integra tecnicas de recuperacion de informacion y procesamiento de lenguaje, junto con sus metricas de evaluacion y el uso de Telegram como plataforma de interaccion con el usuario. La obtencion de resultados permite identificar las fortalezas y debilidades de cada enfoque sin asumir que una tecnica sera mejor que le otra.

De esta manera, al combinar todos estos factores se puede construir un prototipo funcional que permita a los usuarios realizar preguntas y recibir respuestas en espaÃ±ol sobre peliculas.

II. Estado del arte

III. Metodologías para el desarrollo de aplicaciones de Machine Learning

El desarrollo de aplicaciones basadas en aprendizaje automático requiere un proceso organizado que permita abordar las diferentes actividades involucradas en su construcción, desde la identificación del problema hasta la implementación y mantenimiento de la solución. Una de las metodologías propuestas para orientar este proceso es CRISP-ML(Q), cuyo propósito es establecer un modelo de desarrollo para aplicaciones de Machine Learning que incorpore actividades de aseguramiento de calidad durante todo su ciclo de vida.

De acuerdo con la explicación presentada por DataCamp [1], el ciclo de vida puede organizarse en las siguientes seis etapas: planificación, preparación de datos, ingeniería de modelos, evaluación del modelo, implementación del modelo y supervisión y mantenimiento. Fases las cuales se toman como referencia para la organización de las actividades de desarrollo del proyecto.

IV. Técnicas de recuperación de información

En el estudio de "Language Models are Few-Shot Learners" [2] mostraron que un modelo de lenguaje de gran tamaño puede resolver tareas a partir de instrucciones o de pocos ejemplos, sin ajustar sus pesos. Por otra parte, en el estudio "Retrieval-augmented generation for knowledge-intensive NLP tasks" [3] combinan un recuperador con un generador para que la respuesta se apoye en documentos externos, mientras que Qwen2.5 [4] ofrece modelos abiertos que siguen instrucciones, incluidas versiones pequeñas que pueden ejecutarse con recursos limitados, como la que se usa aquí.

En cuanto a la recuperacion de informacion, se consideraron dos enfoques principales para este proyecto: TF-IDF y embeddings multilingües:

A. TF-IDF y similitud del coseno

El esquema de ponderación TF-IDF (*Term Frequency - Inverse Document Frequency*) evalúa la importancia de una palabra en un documento considerando tanto sus apariciones locales (TF) como su rareza en toda la colección (IDF). Aunque durante mucho tiempo se consideró una regla puramente heurística y se intentó justificar mediante la Teoría de la Información de Shannon, Robertson [5] demuestra que los enfoques basados en la información presentan serias inconsistencias teóricas, como la dificultad para definir los espacios de eventos probabilísticos o explicar por qué se ignoran las palabras no incluidas en la consulta. En su lugar, el autor prueba que TF-IDF posee un fundamento sólido en los modelos probabilísticos tradicionales de recuperación de información: el componente IDF equivale al peso de relevancia de Robertson-Spärck Jones (RSJ) cuando no hay datos previos de relevancia, mientras que la combinación TF-IDF se justifica mediante el modelo Okapi BM25, donde la frecuencia local (TF) actúa como evidencia probabilística de que el documento trata verdaderamente sobre el tema deseado.

B. Embeddings y recuperación semántica

Los *embeddings* son representaciones numéricas en forma de vectores dentro de un espacio vectorial, diseñados de tal forma que las oraciones con un significado conceptual similar se ubiquen geométricamente cerca unas de otras [6]. Este concepto es el pilar fundamental de la **recuperación semántica**, un enfoque de búsqueda que permite encontrar información o responder consultas evaluando la similitud de significado entre textos, superando las limitaciones de la simple coincidencia exacta de palabras clave.

La recuperación semántica se vuelve más compleja en escenarios multilingües, donde la mayoría de modelos eficaces existen únicamente para el idioma inglés. Para extender

estos *embeddings*a múltiples lenguas, Reimers y Gurevych [7] proponen la **destilación de conocimiento multilingüe** (*multilingual knowledge distillation*). En este método, un modelo "estudiante" multilingüe (como XLM-RoBERTa) se entrena con pares de oraciones traducidas para imitar las representaciones del modelo "profesor" monolingüe (SBERT en inglés) minimizando el error cuadrático medio. Esta técnica logra dos propiedades cruciales para la recuperación semántica translingüe:

1. alinea los espacios vectoriales entre idiomas (haciendo que oraciones equivalentes en diferentes lenguas compartan la misma ubicación vectorial) 2. transfiere la capacidad de medir similitud semántica a más de 50 idiomas sin sufrir del sesgo de idioma característico de otros métodos tradicionales [7].

V. Comparación de los trabajos revisados

TABLE I: Comparación de los trabajos relacionados con el proyecto.

Autor y año	**Enfoque**	**Aporte principal**	**Relación con CineBot**
DataCamp (2023) [1]	Ciclo de vida de Machine Learning	Describe seis etapas para el desarrollo de aplicaciones de aprendizaje automático: planificación, preparación de datos, ingeniería de modelos, evaluación, implementación y supervisión y mantenimiento.	Se toma como referencia para organizar las actividades y etapas de desarrollo del proyecto.
Robertson (2004) [5]	TF-IDF y recuperación de información	Analiza los fundamentos teóricos del IDF y su relación con los modelos probabilísticos tradicionales de recuperación de información.	Fundamenta el uso de TF-IDF para representar los textos de las películas y realizar búsquedas basadas en la coincidencia de términos.
Reimers y Gurevych (2019) [6]	Representación semántica mediante embeddings	Propone Sentence-BERT, un modelo que genera representaciones vectoriales de oraciones y permite comparar su similitud semántica de manera eficiente.	Proporciona los fundamentos para utilizar representaciones vectoriales en la búsqueda de películas según el significado de las consultas.
Reimers y Gurevych (2020) [7]	Embeddings multilingües	Propone un método de destilación de conocimiento para generar representaciones de oraciones comparables entre diferentes idiomas.	Sustenta el uso de embeddings multilingües para procesar consultas en español y relacionarlas semánticamente con las descripciones de las películas.
Brown et al. (2020) [2]	Modelos de lenguaje de gran tamaño	Estudia la capacidad de los modelos de lenguaje para realizar tareas mediante instrucciones y ejemplos, sin necesidad de ajustar sus parámetros para cada tarea.	Aporta fundamentos sobre el uso de modelos de lenguaje para interpretar instrucciones y generar respuestas en lenguaje natural.
Lewis et al. (2020) [3]	Generación aumentada por recuperación (RAG)	Propone combinar un componente de recuperación de información con un modelo generador para producir respuestas apoyadas en documentos externos.	Se relaciona con la integración de recuperación y generación de respuestas, concepto que sirve como referencia para la capa generativa de CineBot.
Qwen Team (2024) [4]	Modelos de lenguaje Qwen2.5	Presenta una familia de modelos de lenguaje con diferentes tamaños y capacidades para seguir instrucciones y generar texto.	Se relaciona directamente con la selección de Qwen2.5-1.5B-Instruct como modelo para generar las respuestas del chatbot.

VI. METODOLOGÃA

A. Fase 1: Comprension de los negocios y los datos

Muchos usuarios pierden mas tiempo buscando que pelicula ver o intentando recordar su nombre que viendola. Para solucionar este problema, se desarrolla CinemaBox, un asistente que recibe una descripcion de una pelicula en español y recomienda las peliculas que mas se parezcan a dicha descripcion de su catalogo, utilizando telegram como plataforma de comunicacion.

1) Objetivo y criterio de éxito: El objetivo de esta fase es mediante la creacion del sistema, poder comparar el uso de TF-IDF, un enfoque mas tradicional, con el modelo preentrenado embeddings multilingues, para de esta manera, ver que resultados arrojan ambos enfoques a la hora de realizar la misma tarea. Para esto, una vez que el usuario haga una consulta, el sistema evalua todo el catalogo y devuleve un listado ordenado por nivel de similitud, en el que en el primer lugar se encuentra la pelicula mas similar. En este caso, el criterio de exito es la posicion en la que ambos metodos colocan la pelicula correcta, las metricas que se tienen en cuenta son Top-1, Recall@5 y MRR, junto con un tiempo de respuesta de menos de un segundo por consulta.

2) Datos: Los datos usados para el sistema provienen del dataset *TMDB 5000 Movie Dataset* [8], este se eligio porque contiene informacion como titulo, sinopsis, genero, palabras clave, y otros atributos de aproximadamente 4,800 peliculas. De estas 4,800, se descartan aquellas que no tienen sinopsis, en este caso, se descartan 6 peliculas, quedando un total de 4,797.

En cuanto a la calidad de los datos, a pesar de ser un dataset que cuenta con toda la informacion necesaria sobre las peliculas para que el sistema pueda tener un catalogo solido, el que este solo cuente con 4,800 peliculas hace que sea considerado un dataset pequeño, por lo que contar con un conjunto de datos tan limitado limita la variedad de resultados que el bot podria ofrecer si contara con un catalogo mas grande. Por otro lado, la informacion esta completamente en ingles, por lo que aunque el usuario puede hacer sus consultas en español, el bot solo puede responderle en ingles, por lo que la experiencia del usuario se ve limitada por la existencia de esta barrera del idioma. Por ultimo, la tercera limitacion con la que cuenta el dataset escogido es que este se trata de una base de datos de Kaggle totalmente estatica, por lo que no hay actualizaciones con nuevas peliculas y el bot no puede recomendar ninguna que halla sido estrenada despues de la fecha de creacion del dataset (27 de septiembre de 2017).

B. Fase 2: Preparación de datos

1) Limpieza y transformación de los datos: En el archivo original, los generos mas que decir simplemente el nombre del genero de la pelicula venian en un formato que incluia informacion adicional como un id, para solucionar este problema, se borra toda esta informacion adicional para consevar solamente el nombre del genero, por ejemplo, "action". De manera similar tambien se corta la fecha de estreno para conservar unicamente el año en ves del dia, mes y año de estreno de la pelicula, se extraen las palabras clave y se hace lo mismo para los esloganes.

```
    def nombres(celda):
        try:
            return [d['name'] for d in json.loads(celda)]
        except Exception:
            return []

    peliculas = crudo.copy()
    peliculas['generos'] = peliculas['genres'].map(nombres)
    peliculas['palabras_clave'] =
        peliculas['keywords'].map(nombres)
    peliculas['anio'] =
        peliculas['release_date'].astype(str).str[:4]
    peliculas['overview'] =
        peliculas['overview'].fillna('').str.strip()
    peliculas['tagline'] =
        peliculas['tagline'].fillna('').str.strip()
```

Codigo 1. Procesamiento y estructuracion del texto de busqueda.

Otro filtro implementado consiste en filtrar las descripciones que no sean utiles para el sistema, ya que hay algunas peliculas que en su descripcion solo cuentan con frases cortas o poco informativas para el bot como "A great movie".

Por ultimo, tambien se eliminaron los posibles duplicados, ya que si una pelicula aparece dos veces en el dataset podria ocasionar que el sistema termine recomendando la misma pelicula dos veces.

```
    n0 = len(peliculas)
    peliculas = (peliculas[peliculas['overview'].str.len() >
        30]
            .drop_duplicates(subset='id').reset_index(drop=True))
    print(f'Peliculas con sinopsis utilizable:
        {len(peliculas)} (descartadas: {n0 -
        len(peliculas)})')
```

Codigo 2.

2) Construcción del texto de búsqueda: La funcion texto_busqueda() se encarga de construir un texto que combine los generos y las sinopsis, ese texto combinado es el que usa el sistema a la hora de

En vez de que el sistema tenga que buscar directamente sobre todos los campos de una pelicula, se construye un texto que combina sus generos con la sinopsis y dicho texto es el que posteriormente usan los metodos de recomendacion TF-IDF y embeddings multilingues.

La funcion ficha se encarga de reunir los campos de una pelicula como titulo, año, generos, puntuacion y un resumen de la sinopsis recortandola si supera los 180 caracteres, esto para que el bot pueda mostrarle al usuario una ficha con la informacion de la pelicula que le esta recomendando, por lo que este texto no es el que se usa para calcular la similitud entre peliculas.

Por otro lado, se construye la variable CORPUS, *peliculas.to_dict('records')* se usa para transformar las filas del dataframe en una lista de diccionarios por pelicula. El sistema puede compara el primer elemento de cada tupla de CORPUS, que es el texto de busqueda, con la consulta del usuario y devuelve el segundo elemento de la tupla, que es la ficha de la pelicula recomendada, estos dos elementos son almacenados en CORPUS de la manera *(texto_de_busqueda, ficha_de_presentacion)*.

```
def texto_busqueda(f):
    return ', '.join(f['generos']) + '. ' + f['overview']

def ficha(f):
    gen = ', '.join(f['generos']) or 'sin genero'
    sinopsis = f['overview']
    if len(sinopsis) > 180:
        sinopsis = sinopsis[:180].rsplit(' ', 1)[0] + '?'
    return f"? {f['title']} ({f['anio']}) ? {gen} ? ?
        {f['vote_average']}\n{sinopsis}"

registros = peliculas.to_dict('records')
CORPUS = [(texto_busqueda(f), ficha(f)) for f in registros]

print(f'{len(CORPUS)} peliculas cargadas\n')
print(CORPUS[0][1])
```

Codigo 3.

C. Fase 3: Ingeniería de modelos

Para el desarrollo del sistema, se implementaron dos enfoques con dos modelos de recuperacion de informacion diferentes, ambos utilizados bajo las mismas condiciones, es decir, con el mismo catalogo de peliculas. Los modelos se implementaron de la siguiente manera:

1) Enfoque 1: TF-IDF: La implementacion de este modelo se hizo siguiendo una serie de pasos, en el que despues de que se importan las librerias necesarias como numpy, counter y re, se definen las palabras vacias o *stopwords*, estas consisten en todas aquellas palabras que aunque son frecuentes, no aportan nada de informacion relevante al sistema, por ejemplo, palabras como *el, la, de, que, etc.*. Esto se hace porque asi el sistema puede centrarse unicamente en las palabras que realmente ayuden a identificar el contenido de la pelicula.

Una vez definidas las palabras vacias, se limpia el texto, con limpiar se hace referencia a: convertir las palabras en minusculas, para que el sistema no trate las diferencias entre mayusculas y minusculas como palabras distintas, eliminar caracteres que no esten permitidos, como los signos de puntuacion y numeros los cuales son reemplazados por espacios. Por ultimo, se separan y filtran las palabras utilizando slip() para dividir el texto en palabras y eliminar aquellas que esten en la lista de STOPWORDS o que solo cuenten con un caracter.

```
def limpiar(texto):
    texto = texto.lower()
    texto = re.sub(r'[^a-zaeiouun ]', ' ', texto)
    tokens = texto.split()
    return [t for t in tokens if t not in STOP and len(t) >
        1]
```

Codigo 4.

El siguiente paso es construir el indice TF-IDF, es decir, la estructura de datos que se construye para que el sistema pueda comparar la consulta que hace el usuario con las peliculas que tiene en su catalogo y asi encontrar la que mas se parezca. Si el sistema tiene una cantidad x de peliculas, el codigo procesa los textos de cada una de ellas y a partir de eso construye el vocabulario indentificando las palabras que aparecen en el catalogo, utilizando *word2idx* para asignarle a cada palabra una posicion numerica mientras que la variable *v* representa el numero total de palabras unicas del vocabulario. De esta manera, se puede representar a cada pelicula mediante un vector que tenga el mismo numero de posiciones que de palabras en el vocabulario.

```
def construir_indice(documentos):
    tokenizados = [limpiar(d) for d in documentos]
    N = len(tokenizados)

    # Vocabulario: todas las palabras unicas del corpus
    vocab = sorted({t for doc in tokenizados for t in doc})
    word2idx = {w: i for i, w in enumerate(vocab)}
    V = len(vocab)
```

Codigo 5.

La siguiente etapa es calcular los pesos TF-IDF asignandole un valor a cada palabra segun su frecuencia en cada pelicula y que tan comun es en el catalogo. Para poder lograr esto, lo primero que se hace es calcular el componente IDF (Inverse Document Frequency). La formula implementada es:

$$\text{idf}(t) = \ln\left(\frac{N+1}{\text{df}(t)+1}\right) + 1$$

Se le suma uno al numerador y al denominador para evitar la division por cero, ya que sin sumar +1, el codigo evaluaria una palabra que no aparece en ninguna pelicula del catalogo, df=0, por lo que la formula seria N/0, lo que generaria un error matematico. Tambien se le suma +1 al resultado final porque sin esa suma, si una palabra aprece en todas las peliculas del catalogo (df=N), el resultado de la formula seria 0, lo que no es deseable porque haria que esa palabra fuese ignorada por el sistema en la busqueda.

En el bucle se usa *set(doc)*para contar cada palabra una sola vez por pelicula al calcular df.

```
    # IDF de cada palabra
    df = np.zeros(V)
    for doc in tokenizados:
        for w in set(doc):
            if w in word2idx:
                df[word2idx[w]] += 1
    idf = np.log((N + 1) / (df + 1)) + 1 # +1 evita division
        por cero
```

Codigo 6.

Por ultimo, se calcula TF (Term Frequency), este concepto hace referencia a la frecuencia de cada palabra en cada pelicula, la formula implementada es:

$$\text{tf}(t,d) = \frac{\text{count}(t,d)}{|d|}$$

Se multiplica el resultado de TF por IDF, quedando de la forma:

$$\text{tf-idf}(t,d) = \text{tf}(t,d) \cdot \text{idf}(t)$$

Una palabra que aparece muchas veces en una pelicula, es decir, con una frecuencia alta, pero que al mismo tiempo, no esta presente en demasiadas peliculas recibe un peso mayor.

Posteriormente, se crea la matriz TF-IDF en la que representa cada pelicula mediante un vector de pesos TF-IDF. En el codigo, este paso se implemento como:

```
    # Matriz TF-IDF: una fila por pelicula, una columna por
        palabra
    matriz = np.zeros((N, V), dtype=np.float32) # float32:
        mitad de memoria
    for i, doc in enumerate(tokenizados):
        cnt = Counter(doc)
        total = len(doc) or 1
        for w, c in cnt.items():
            if w in word2idx:
                matriz[i, word2idx[w]] = (c / total) *
                    idf[word2idx[w]]
```

Codigo 7.

La normalizacion de los vectores se hace para que cada vector de pelicula tenga una longitud de 1, es decir, que todos los vectores esten en la misma escala. Esto se hace para ajustar la escala de los vectores y que asi, la comparacion se centre en la distribucion de los pesos en lugar de depender solo de la magnitud. La matriz resultante se guarda como el indice TF-IDF que utilizara el bot.

```
    # Normalizar cada fila (L2) para que la comparacion sea
        por direccion, no magnitud
    normas = np.linalg.norm(matriz, axis=1, keepdims=True)
    normas[normas == 0] = 1
    matriz /= normas

    return matriz, idf, word2idx
```

Codigo 8.

La funcion *vectorizar* se encarga de convertir una consulta de un usuario en un vector, para eso, aplica el mismo preprocesamiento y los mismos pesos IDF que se calcularon con el catalogo de peliculas. Una vez hecho esto, se le asigna un peso a cada palabra conocida, teniendo en cuenta que solo se consideran las palabras que existen en el vocabulario del catalogo, asi que si una consulta contiene palabras que no aparecen en dicho vocabulario, esas palabras no le aportan peso al vector. Por ultimo, se normaliza la consulta para que de esta manera, tanto las peliculas como las consultas utilicen el mismo espacio vectorial y se les pueda comparar entre si:

```
def vectorizar(texto, idf, word2idx):
    tokens = limpiar(texto)
    vec = np.zeros(len(word2idx), dtype=np.float32)
    cnt = Counter(tokens)
    total = len(tokens) or 1

    for w, c in cnt.items():
        if w in word2idx:
```

```
        vec[word2idx[w]] = (c / total) * idf[word2idx[w]]

    norma = np.linalg.norm(vec)
    if norma > 0:
        vec /= norma
    return vec
```

Codigo 9.

A continuacion, se construye el indice del catalogo, este se construye una vez al iniciar el motor, en lugar de volverse a calcular todos los vectores cada vez que llega una consulta.

textos contiene los textos utilizados para calcular los vectores, *fichas* conserva la informacion que se mostrara al usuario cuando el sistema le recomiende una pelicula, *matriz_tfidf* almacena los vectores normalizados de todas las peliculas, *idf* contiene los pesos de las peliculas y *word2idx* permite localizar cada palabra en el vocabulario.

```

textos = [par[0] for par in CORPUS]
fichas = [par[1] for par in CORPUS]

matriz_tfidf, idf, word2idx = construir_indice(textos)

print(f'Motor TF-IDF listo ? {len(textos)} peliculas,
     {len(word2idx)} palabras en vocabulario')
```

Codigo 10.

Para encontrar las tres peliculas mas similares se usa la funcion *buscar_tfidf*, esta se encarga de vectorizar la consulta convirtiendo el texto del usuario en un vector normalizado, luego *matriz_tfidf @ vec* calcula el producto que hay entre el vector de consulta y el de cada pelicula, al estar estos vectores normalizados, el resultado termina siendo la similitud del coseno:

$$\text{similitud} = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|}$$

al estar los vectores normalizados pasa a ser:

$$\text{similitud} = \vec{q} \cdot \vec{d}$$

Los resultados, es decir, las peliculas, se ordendan de mayor a menor con *np.argsort(-similitudes)*, y por ultimo, con *[:k]* se devuelven los indices de las tres peliculas con las puntuaciones mas altas junto con sus similitudes.

```

def buscar_tfidf(consulta, k=3):
    vec = vectorizar(consulta, idf, word2idx)
    similitudes = matriz_tfidf @ vec
    top = np.argsort(-similitudes)[:k]
    return top, similitudes[top]
```

Codigo 11.

El umbral de similitud se aplica para permitir que el bot no devuelva automaticamente una pelicula cuando la similitud maxima es demasiado baja, asi que si la puntuacion de la pelicula mas cercana es igual o superior a 0.05, se devuelve su ficha, pero si esta por debajo el bot muestra un mensaje indicando que no encontro peliculas similares.

El valor de 0.05 es solo una configuracion inicial la cual puede ajustarse conforme se vaya probando el sistema.

```

UMBRAL_TFIDF = 0.05

def responder_tfidf(consulta, umbral=UMBRAL_TFIDF):
    idxs, sims = buscar_tfidf(consulta)
    if sims[0] >= umbral:
        return fichas[idxs[0]]
    return 'No encontre peliculas parecidas. Intenta
         reformular tu consulta o escribe /ayuda.'
```

Codigo 12.

Por ultimo, se ejecutaron una pruebas las cuales permiten verificar el comportamiendo del motor TF-IDF con diferentes tipos de entradas entre las que se encuentra una consulta en ingles y en español sobre el mismo tema (robots), otra en ingles sobre dinosaurios y un parque tenamico y una consulta que esta fuera del contexto de peliculas:

```

print("Probando el motor TF-IDF:\n")
pruebas = [
    'a robot that falls in love',
    'una pelicula de robots que se enamoran',
    'dinosaurs escape from a theme park',
    'receta de empanadas',
]
for p in pruebas:
    print(f' Usuario: {p}')
    print(f' Bot: {responder_tfidf(p)}\n')
```

Codigo 13.

2) Enfoque 2: Embeddings multilingües: La diferencia entre este segundo enfoque con el primero es que como ya se explico, con TF-IDF se representa las peliculas mediante pesos que estan asociados a palabras y la busqueda depende principalmente de que existan palabras en comun, mientras que por otro lado, con Embeddings se representa las peliculas mediante vectores mas densos que permiten encontrar los textos relacionandonos por su significado, es decir, que aunque no compartan las mismas palabras, si el significado es similar, el sistema puede encontrar la pelicula correcta, incluso si se utilizan palabras que esten en otro idioma.

Para la implemantacion de embeddings multilingues, se usaron las librerias torch para ejecutar operaciones de aprendizaje profundo y sentence-transformes para cargar modelos preentrenados de embeddings. Con TF-IDF, se necesita calcular manualmente todas las frecuencias de las palabras como con TF-IDF mientras que con embeddings, el modelo ya esta preentrenado y solo se necesita cargarlo e implementarlo para que el sistema pueda convertir los textos en vectores.

Para eso, el primer paso es seleccionar el dispositivo de ejecucion, se comprueba si hay una GPU disponible, si es asi, se ejecuta el modelo en la GPU con CUDA, de lo contrario, se ejecuta en la CPU utilizando el procesador. Con esto lo que se busca es poder aprovechar la GPU cuando este disponible para acelerar los calculos, ya que los modelos de embeddings suelen ser mas pesados y requieren mas recursos de computo que TF-IDF.

```
import torch
from sentence_transformers import SentenceTransformer

dispositivo = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Dispositivo:', dispositivo)

emb_model =
     SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2',
     device=dispositivo)
```

Codigo 14.

Una vez hecho esto, se puede cargar el modelo preentrenado, *paraphrase* hace referencia a que como ya se comento antes, el modelo puede generar representaciones de frases para poder comparar su significado, *multilingual* permite trabajar con multiples idiomas y *MiniLM* hace referencia a la arquitectura del modelo en la que se procura que sea mas ligera que otros modelos mas grandes, mientras que el numero 12 hace referencia al numero de capas.

Este modelo recibe un texto y genera un vector numerico que representa el significado del mismo, asi si dos frases usan palabras diferentes pero tienen el mismo significado, el modelo puede representarlas mediante vectores cercanos, aun si las palabras estan escritas en idiomas diferentes.

```
emb_model =
     SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2',
     device=dispositivo)
```

Codigo 15.

El siguiente paso es convertir todas las peliculas en embeddings, con encode se convierte cada texto en un vector numerico, sus parametros utilizados son:

- textos: Consiste en una lista de textos de las peliculas que se convertiran en vectores
- batch_size: Se encarga de procesar los textos en grupos de hasta 64 para organizar el calculo.
- normalize_embeddings: Con este parametro se normalizan los vectores para que tengan una longitud igual a 1.
- show_progress_bar: Este parametro se usa para mostrar una barra de progreso mientras se generan los embeddings.

El resultado final se guarfa en la variable *matriz_emb*, esta es la matriz que contiene los emveddings de todas las peliculas, en este cada pelicula esta representada por cada fila de la matriz, mientras que cada columna representa una dimension del embedding y cada valor es un componente numerico del embedding. Las dimensiones son caracteristicas aprendidas por el modelo, mientras que los valores de las peliculas se calculan una sola vez y se conservan para las busquedas posteriores, al igual que con TF-IDF.

```
matriz_emb = emb_model.encode(textos, batch_size=64,
    normalize_embeddings=True, show_progress_bar=True)
```

Codigo 16.

La funcion *buscar_emb* recibe la consulta del usuario y se encarga de buscar las peliculas que tengan la mayor similitud con la consulta.

La consulta se convierte en un embedding, para esto, primero tiene que convertirse en un vector, asi, cuando el usuario hace una consulta el modelo genera un embedding que represente el significado de esa frase ingresada por el usuario.

Las similitudes se calculan mediante el producto entre la matriz de los embeddings de la consulta del usuario y la matriz de los embeddings de todas las peliculas, este resultado equivale a la similitud del coseno porque los vectores estan normalizados. Una consulta en español puede hacer que el sistema le recomiende al usuario una pelicula con una sinopsis en ingles aunque por la diferncia de idioma, no halla ninguna coincidencia entre las palabras.

Las peliculas mas similares se seleccionan con *np.argsort(-similitudes)*, esta instruccion se encarga de ordenar las peliculas segun su similitud y selecciona las primeras k (3) peliculas con las puntuaciones mas altas.

Esta funcion termina retornando dos elementos, top, que contiene los indices de las tres peliculas seleccionadas, y similitudes[top], que contiene las puntuaciones de similitud de esas peliculas con la consultada por el usuario.

```
def buscar_emb(consulta, k=3):
    vec = emb_model.encode([consulta],
        normalize_embeddings=True)[0]
    similitudes = matriz_emb @ vec
    top = np.argsort(-similitudes)[:k]
    return top, similitudes[top]
```

Codigo 17.

El umbral de similitud se define estableciendo un valor inicial (0.30) para el motor de embeddings, este valor se utiliza posteriormente en el codigo en el comportamiento del bot.

Por otro lado, se comprueba que el motor esta listo para mostrar las dimensiones de la matriz de embeddings.

```
UMBRAL_EMB = 0.30

print(f'Motor de embeddings listo ? matriz
    {matriz_emb.shape}')
```

Codigo 18.

Para realizar las pruebas se hacen las mismas consultas de prueba que las que ya se usaron cuando se probo TF-IDF:

```
for p in ['a robot that falls in love', 'una pelicula de
    robots que se enamoran','dinosaurs escape from a theme
    park', 'receta de empanadas']:
    idxs, sims = buscar_emb(p)
    print(f' Usuario: {p}')
    print(f' Top-1: {peliculas.loc[idxs[0], "title"]}
        (similitud {sims[0]:.2f})\n')
```

Codigo 19.

a) LLM: Al enfoque 2 con embedding se le implemento esta capa extra que consiste en un modelo de lenguaje LLM, esto se incluyo principalmente porque aunque el sistema es funcional, ya que es capaz de encontrar las peliculas, el problema que tiene es que estos resultados que da se devuelven en forma de basica con una lista de peliculas con sus puntuaciones de similitud con lo que busco.

La ventaja que ofrece el LLM es que el sistema pueda tomar las peliculas encontradas y en base a ellas, generar una respuesta mas elaborada y natural para el usuario.

La implementacion en el codigo se llevo de la siguiente manera, en la que despues de importar y cargar el modelo Qween, se definen las instrucciones del asistente y se crea la funcion *asistente*. En esta, se recibe la consulta del usuario y se genera una respuesta mediante el LLM, despues de comprobar que el modelo esta cargado, el asistente usa la funcion *buscar_emb* para recuperar las 3 peliculas que se determinaron como las mas similares a lo que busca el usuario, luego, se construye el contexto que el cual contiene la informacion relevante sobre las peliculas como el titulo, año, generos y los primeros 300 caracteres de la sinopsis.

Esto es importante porque Qween necesita poder recibir la información sobre las peliculas para de esa manera, poder elaborar una recomendacion en base a los resultados que encontro el motor de embeddings sin tener que realizar una nueva busqueda en todo el catalogo.

En la variable de texto_chat, se convierten los mensajes al formato del modelo, ya que para funcionar, en este caso necesita seguir la plantilla de conversacion que utiliza Qween *apply_chat_template*. Posteriormente, se tokeniza la entrada transformando el texto en numeros que el modelo puede procesar y los resultados se devuelven como tensonres de pyTorch, que es la libreria que se utiliza para ejecutar el modelo, con *.to(dispositivo)* se colocan estos tensores en el dispositivo donde esta cargado el modelo.

La salida se genera con *llm_model.generate*, y se decodifica con *llm_tok.decode* para convertir los numeros de vuelta a texto, que es lo que finalmente se devuelve al usuario, el modelo devuelve una secuencia de tokens que contiene tanto la entrada original como los tokens generados.

```
def asistente(consulta, k=3):
    if llm_model is None:
        return 'El asistente con LLM no esta activado
            (USE_LLM = False).'
    idxs, sims = buscar_emb(consulta, k)
    contexto = '\n'.join(
        f"- {peliculas.loc[i, 'title']} ({peliculas.loc[i,
            'anio']}), generos: {', '.join(peliculas.loc[i,
            'generos'])}. "
        f"Sinopsis: {peliculas.loc[i, 'overview'][:300]}"
        for i in idxs)
    mensajes = [
        {'role': 'system', 'content': SYSTEM_LLM},
        {'role': 'user', 'content': f'Consulta del usuario:
            {consulta}\n\nPeliculas
            encontradas:\n{contexto}\n\n'
                                'Recomienda las que mejor
                                    encajan y explica en una
                                    frase por que.'},
    ]
    texto_chat = llm_tok.apply_chat_template(mensajes,
        tokenize=False, add_generation_prompt=True)
    entrada = llm_tok(texto_chat,
        return_tensors='pt').to(dispositivo)
```

```
    with torch.no_grad():
        salida = llm_model.generate(**entrada,
            max_new_tokens=200, do_sample=False,
                                pad_token_id=llm_tok.eos_token_id)
    return llm_tok.decode(salida[0,
        entrada['input_ids'].shape[1]:],
        skip_special_tokens=True).strip()
```

Codigo 20.

Por ultimo, se realizan pruebas para revisar el comportamiento de esta parte del sistema. En este caso, se usan las 3 primeras consultas del conjunto de descripciones en español y una consulta adicional sobre las peliculas de terror con casas embrujadas. Tambien se mide cuanto tiempo se demora con cada respuesta usando time.time().

```
if USE_LLM:
    for q in es_q[:3] + ['una pelicula de terror con casas
        embrujadas']:
        t0 = time.time()
        r = asistente(q)
        print(f'Usuario: {q}\nCinemaBox: {r}\n({time.time() -
            t0:.1f} s)\n' + '-' * 60)
```

Codigo 21.

D. Fase 4: Evaluación del modelo

La evaluacion del modelo se realiza inicialmente normalizando los titulos de las peliculas, esto para poder identificarlas correctamente en el catalogo ya que hay titulos que cuentan con caracteres especiales como mayusculas, espacios o signos de puntuacion. Luego, se le asigna una posicion numerica a cada pelicula comenzando desde cero, esto para posteriormente identificar cual es la pelicula correcta dentro de los resultados obtenidos.

Conjuntos de consulta. Los conjuntos de consulta se utilizan para probar el sistema y ver que tan bien funciona. El sistema cuenta con 3 conjuntos: A, B y C, y cada uno representa una manera diferente en la que se le puede preguntar al sistema por una pelicula:

- Conjunto A (usando el eslogan de la pelicula (tagline)): En este conjunto, la consulta se hace tomando el eslogan de una pelicula del dataset y utilizandola como si esa fuera la consulta que hizo el usuario.
- Conjunto B (usando las cinco primeras palabras clave): En el conjunto B, se toman las 5 primeras palabras claves que esten asociadas a una pelicula y estas se utilizan como si fueran la consulta.
- Conjunto C (usando las descripciones escritas en español): Por ultimo, en el conjunto C en ves de usar un campo del dataset, se redacta una descripcion de una pelicula en español y el sistema la relaciona con su titulo.

El uso de estos tres conjuntos resulta util ya que permite probar el comportamiento del sistema en 3 diferentes escenarios:

```
# Conjunto A: eslogan
cand = peliculas[peliculas['tagline'].str.len() > 10]
m = cand.sample(min(N_EVAL, len(cand)), random_state=42)
CONJUNTOS = {'A. Eslogan': (m['tagline'].tolist(),
    m['pos'].to_numpy())}

# Conjunto B: palabras clave (las 5 primeras de TMDB)
cand = peliculas[peliculas['palabras_clave'].map(len) >= 3]
m = cand.sample(min(N_EVAL, len(cand)), random_state=42)
CONJUNTOS['B. Palabras clave'] = ([' '.join(k[:5]) for k in
    m['palabras_clave']], m['pos'].to_numpy())

# Conjunto C: descripciones en espanol escritas a mano
CONSULTAS_ES = [
    ('Un barco gigante choca con un iceberg y una chica rica
        se enamora de un chico pobre', 'Titanic'),
    #...mas descripciones escritas a mano...
]
```

Codigo 22.

Una vez hecho esto, se comprueban que los titulos existan en el catalogo, para eso, se recorre las descripciones y sus titulos, si se encuentra la pelicula en el catalogo, se guarda la descripcion y la posicion de la pelicula correcta, de no encontrarla, se guarda el titulo de la pelicula que no se encontro en faltan y se omite esa consulta

```
pos_por_titulo = (peliculas.sort_values('vote_count',
    ascending=False)
            .drop_duplicates('norm').set_index('norm')['pos'])

es_q, es_t, faltan = [], [], []

for q, titulo in CONSULTAS_ES:
    n = normalizar_titulo(titulo)
    if n in pos_por_titulo.index:
        es_q.append(q)
        es_t.append(int(pos_por_titulo[n]))
    else:
        faltan.append(titulo)

CONJUNTOS['C. Espanol (manual)'] = (es_q, np.array(es_t))
print(f'Conjunto C: {len(es_q)} consultas usables. No estan
    en el catalogo (omitidas): {faltan}\n')
```

Codigo 23.

Una vez que estan construidos los conjuntos, se necesita ejecutar ambos modelos sobre las mismas consultas, para eso, se definien dos funciones, una para TF-IDF *puntajes_tfidf* y otra para embeddings *def puntajes_emb*, estas funciones reciben una lista de consultas y calculan la similitud entre cada consulta y todas las peliculas del catalogo, devolviendo una matriz de puntuaciones de similitud, en el que cada valor indica que tan similar es una consulta a una pelicula. Se crean dos funciones diferentes porque se necesita evaluar los dos metodos por separado, pero bajo las mismas condiciones, para que la comparacion sea justa.

Una vez hecho esto, se crea un diccionario *METODOS = 'TF-IDF': puntajes_tfidf,'Embeddings': puntajes_emb* en el que se relaciona el nombre de cada metodo con su funcion de evaluacion, lo que permite que se pueda ejecutar ambos modelos mediante un mismo ciclo sin tener que escribir el procedimiento dos veces.

La funcion *posiciones* tiene como objetivo principal el poder determinar en que posicion queda la pelicula correcta dentro del ranking que genera cada modelo, en el que *s* es la matriz de puntuaciones y cada fila es una consulta, cada columna una pelicula y cada valor la similitud entre ambos, mientras que *objetivos* contiene las posiciones de las peliculas correctas para cada consulta.

Finalmente, la evaluacion de los conjuntos de consulta se hace recorriendo los conjuntos A,B y C, y aplica tanto TF-IDF como embeddings a las consultas de cada conjunto antes de calcular las metricas de evaluacion ya escogidas:

- Top-1: Calcula que proporcion de consultas obtuvo la pelicula correcta en la primera posicion
- Recall@5: Calcula la proporcion de consultas en las que la pelicula correcta aparece entre las primeras cinco posiciones
- Recall@10: El recall@10 realiza el mismo calculo que el anterior, pero considerando las diez primeras posiciones
- MRR: Mean Reciprocal Rank calcula el promedio del inverso de la posicion de la pelicula correcta.Cuanto mas cerca esta la pelicula correcta de la primera posicion, mayor sera su valor al MRR.

Estas metricas se utilizan porque permiten saber si el sistema recomendo la pelicula correcta en primer ligar o si por el contraio, el usuario tendria que revisar varias opciones antes de encontrarla. Los resultados se guardan en el dataframe *resultados*.

```
filas, PUNTAJES, POSICIONES = [], {}, {}
for conjunto, (consultas, objetivos) in CONJUNTOS.items():
    for metodo, fn in METODOS.items():
        S = fn(consultas)
        pos = posiciones(S, objetivos)
        PUNTAJES[(conjunto, metodo)], POSICIONES[(conjunto,
            metodo)] = S, pos
```

```
        filas.append({'Conjunto': conjunto, 'Metodo': metodo,
            'n': len(consultas),
                'Top-1': (pos <= 1).mean(), 'Recall@5':
                    (pos <= 5).mean(),
                'Recall@10': (pos <= 10).mean(), 'MRR': (1
                    / pos).mean()})

resultados = pd.DataFrame(filas)
```

Codigo 24.

La medicion del tiempo de consulta se hace para medir cuanto tarda cada metodo en realizar la busqueda, para eso, se define la funcion *ms_por_consulta* que recibe como parametro el metodo de busqueda que se desea evaluar, el procedimiento consiste en registrar el instante inicial, recorrer las primeras 30 consultas y se ejecuta la busqueda para cada consulta, luego, con *time.time() - t0* se calcula cuanto tiempo transcurrio duarnte todas las busquedas, luego, se divide el tiempo entre el numero de consultas y se multiplica con 1000 para convertir el resultado de segunfos a milisegundos.

En *latencia* se calcula el tiempo promedio de busqueda de cada motor, y se agrega una columna a la tabla de resultados con la latencia correspondiente a cada metodo *resultados['ms/consulta'] = resultados['Método'].map(latencia)*.

E. Fase 5: Implementación del modelo

El sistema se despliega utilizando Telegram para gestionar los mensajes que envian los usaurios, para eso, se implementaron las librerias necesarias para preparar el entorno y que Python pueda conectarse con Telegram, sobretodo la libreria *asyncio* ya que es la que permite poder ejecutar operaciones asincronas en Python y asi asegurar que el bot pueda gestionar estas tareas sin tener que esperar a que cada una de ellas termine para poder ejecutar la sifuiente.

En la variable *BIENVENIDA* se almacenan los mensajes que el usuario recibe por parte del bot a la hora de iniciar una conversacion, mientras que en la variable *AYUDA* se almacenan los ejemplos de consultas que el usuario puede hacerle al bot con el objetivo de orientarlo sobre como describir la pelicula que desea encontrar. En *metodo_usuario* se almacena el metodo de recomendacion que el usuario seleccione.

Posteriormente, se procede a crear los manejadores de comandos y mensajes, siendo esta la parte en la que se define la logica que debe seguir el bot ante las acciones del usuario, estas funciones se ejecutan cuando Telegram recibe un mensaje como por ejemplo */start* o */asistente*.

Este ultimo permite utilizar la generacion de respuestas mediante un modelo de lenguaje LLM, funcion que permite al usuario recibir una respuesta con lenguaje mas natural. Primero se obtiene la consulta del usuario y despues de verificar que esta no este vacia, se ejecuta el asistente mediante la funcion *asyncio.to_thread*, la cual realiza la recuperacion de las peliculas mediante embedding y utiliza los resultados obtenidos como contexto para que la LLM pueda generar una respuesta mas completa.

```
async def cmd_asistente(update: Update, context:
    ContextTypes.DEFAULT_TYPE):
   consulta = ' '.join(context.args)
   if not consulta:
      await update.message.reply_text('Escribe tu consulta
          después del comando. Ejemplo:\n'
                              '/asistente una película de
                                  robots que se enamoran')
      return
   await update.message.reply_text('Pensando...')
   respuesta = await asyncio.to_thread(asistente, consulta)
   await update.message.reply_text(respuesta)
```

Codigo 25.

Tambien se le permite al usuario poder procesar mensajes de texto libre sin necesidad de estar limitado a escoger alguno de los comandos con la funcion *async def on_mensaje*, la cual se encarga de registrar el mensaje del usuario y procesarlo mediante el metodo de recomendacion que este haya seleccionado, devolviendo la ficha de la pelicula recomendada.

Se construye la aplicacion de Telegram mediante *app = ApplicationBuilder().token(TOKEN).build()*, en donde se registran los manejadores. Se utilizan tres elementos clave, *ApplicationBuilder()*, que inicia la configuracion de la aplicacion, *token(TOKEN)*, que establece el token de autenticacion que fue proporcionado por Telegram con BotFather y *build()*, que construye la aplicacion con la configuracion ya establecida.

Los comandos se registran con *app.add_handler()*, cada *CommandHandler* relaciona un comando con su respectiva funcion, mientras que los registros de los mensajes de texto libre se hacen con *MessageHandler*.

Por ultimo, el bot se ejecuta con *app.run_polling()*, el cual se encarga de poner en funcionamiento el chatbot.

F. Fase 6: Supervisión y mantenimiento

Uno de los componentes del sistema mas importantes es el modelo de recuperacion, ya que este es el encargado de identificar las peliculas que se le van a recomendar al usuario, por eso, es importante garantizar su calidad y correcto funcionamiento, para esto, se considera verificar los siguientes aspectos:

- Calidad de las recomendaciones: Revisar si las peliculas recomendadas por el bot correspondan con lo pedido por el usuario.
- Comparacion entre metodos: Analisar las diferencias que hay entre los resultados mediante TF-IDF y embeddings, sobretodo cuando se hacen consultas en español.
- Consultas fuera del dominio: Verificar como es el comportamiento del modelo cuando recibe consultas que no esten relacionadas con peliculas
- Tiempo de respuesta: Medir cuanto tiempo le toma al sistema procesar la consulta y procurar que dicho tiempo no se exceda demasiado, ya que esto podria afectar la experiencia del usuario.

Para supervisar el funcionamiento del chatbot en Telegram, se verifica que los componentes del sistema funcionen correctamente. Esta supervision incluye:

- Recepcion de consultas, para verificar que el bot reciba correctamente los mensajes enviados por los usuarios.
- Funcionamiento de los metodos de recuperacion: Se comprueba que los comandos */start*, */ayuda*, */metodo* y */asistente* ejecuten correctamente.
- Seleccion del metodo de recuperacion: Verificar que el bot utilice TF-IDF, embeddings o ambos metodos dependiendo la elegida por el usuario.
- Generacion de recomendaciones: Comprobar que las consultas sean procesadas de manera correcta y que se le presenten al usuario las peliculas recomendadas.
- Generacion de respuestas con LLM: Se verifica que el comando */asistente* utilice los resultados de embeddings para generar una respuesta redactada en lenguaje natural.

VII. RESULTADOS

A continuacion, en la Tabla II y la Fig. 1 se presentan los los resultados de las metricas de rendimiento evaluadas.

TABLE II
RESULTADOS CUANTITATIVOS DE EVALUACIÓN POR CONJUNTO DE PRUEBA Y MÉTODO.

Conjunto	Método	n	Top-1	Recall@5	Recall@10	MRR	ms/consulta
A. Eslogan	TF-IDF	1000	0.044	0.089	0.121	0.070	35.06
	Embeddings	1000	0.049	0.107	0.136	0.079	14.86
B. Palabras clave	TF-IDF	1000	0.137	0.285	0.350	0.209	35.06
	Embeddings	1000	0.124	0.242	0.303	0.189	14.86
C. Español (manual)	TF-IDF	39	0.000	0.128	0.128	0.048	35.06
	Embeddings	39	**0.256**	**0.513**	**0.641**	**0.383**	**14.86**

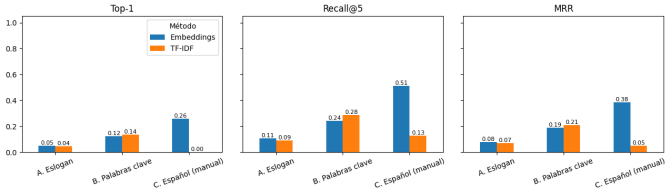


Fig. 1. Comparación de las métricas de evaluación.

A. Resultados por conjuntos de consulta.

- Conjunto A: En el conjunto A se ve que ambos metodos tienen un desempeño que podria considerarse bajo, el MRR es de 0.079 para embeddings y 0.070 con TF-IDF, lo que indica que los esloganes no son un buen recurso para que el sistema pueda recomendar una pelicula porque son solo frases que rara vez describen la trama de la pelicula, por lo que el sistema no obtiene suficiente informacion. La diferencia de MRR a favor de embeddings es de 0.009.
- Conjunto B: En el conjunto B, el modelo TF-IDF supera a embeddings con un MRR de 0.209 en comparacion con el 0.189 de embeddings, lo que indica que las palabras clave son un recurso bastande util ya que estas suelen aparecer en la sinopsis de la pelicula, por lo que el sistema puede encontrar las peliculas mas facilmente. En este caso, la diferencia entre embeddings y TF-IDF es de -0.020.
- Conjunto C: Se puede observar que el modelo de embeddings supera a TF-IDF en el conjunto C, que es el conjunto de consultas en español. Las metricas registraron un Recall@5 de 51.3% y un MRR de 0.383, mientras que TF-IDF obtuvo un Recall@5 de 12.8% y un MRR de 0.048. Por otro lado, las mediciones de latencia confirman que realizar una ejecucion sobre GPU (CUDA) permite que se procese cada consulta con un promedio de 14.86ms mediante la busqueda por producto matricial sobre embeddings, mientras que TF-IDF requiere un promedio de 35.06ms por consulta, lo que representa una diferencia de 2.36 veces mas rapido para el modelo de embeddings.

B. Análisis de recomendaciones.

Se realizo un analisis en el que se compararon las recomendaciones dadas por ambos metodos para una misma consulta "una pelicula de robots que se enamoran", esto con el fin de observar que respuestas dan cada uno.

En el caso de TF-IDF, la primera pelicula que el sistema recomendo fue Sky Captain and the world of tomorrow de 2004, la descripcion de la pelicula menciona ataques de robot gigantes, mientras que la segunda pelicula recomendada fue Heartbeeps de 1981, la cual es una pelicula de una historia de amor entre dos robots, y por ultimo, el sistema recomendo la pelicula Surrogates (2009), una pelicula sobre la vida en un mundo donde los humanos son reemplazados por réplicas. Por otro lado, el modelo de embeddings recomendo como primera pelicula Heartbeeps (1981), seguida de la pelicula The Lovers (2015) y A.I. artifial intelligence (2001), ambas peliculas sobre historias de amor entre robots, lo que indica que en el modelo embedding existe una mayor relacion entre la intencion del usuario con la consulta y la pelicula recomendada como primera opcion.

Estas diferencias se pueden apreciar tambien en los valores de similitud que el sistema le asigno a cada una de las recomendaciones dadas, ambos modelos recomendaron la pelicula Heartbeeps (1981), con la diferencia de que TF-IDF le asigno un puntaje de 0.30 mientras que embeddings le asigno 0.69. Aunque estos puntajes no son equivalentes en los dos modelos, ya que cada uno cuenta con sus propias escalas de similitud, permiten observar el orden de relevancia que se establecio por cada modelo.

En general, el ejemplo permitio ver que ambos metodos permiten recuperar peliculas que esten relacionadas con los terminos que el usuario utiliza en su consulta, pero son diferentes a la hora de diferenciar la intencion y significado de una descripcion, siento embeddings el que logro ubicar primero la pelicula que coincide mas con la descripcion que fue dada por el usuario.

TABLE III
COMPARACIÓN DE RECOMENDACIONES PARA LA CONSULTA "UNA PELÍCULA DE ROBOTS QUE SE ENAMORAN".

Posición	TF-IDF	Embeddings
1	Sky Captain and the World of Tomorrow (0.31)	Heartbeeps (0.69)
2	Heartbeeps (0.30)	The Lovers (0.62)
3	Surrogates (0.26)	A.I. Artificial Intelligence (0.59)

C. Tiempo de respuesta

En cuanto a los tiempos de respuesta, TF-IDF registro en promedio 35.064ms por consulta mientras que Embeddings registro 14.857ms, valores que se obtuvieron teniendo en cuenta las condiciones de prueba establecidas, en las que TF-IDF se ejecuto con CPU y Embeddings en GPU. Para complementar el analisis del tiempo de respuesta, se realizo una medicion extra en la que se evaluo el modelo Embeddings en CPU con el fin de poder hacer una comparacion mas justa, fue asi que se lograron obtener los siguientes resultados:

TABLE IV
COMPARACIÓN DE LOS TIEMPOS DE RESPUESTA SEGÚN EL MÉTODO Y EL HARDWARE DE EJECUCIÓN.

Método	Hardware	Tiempo (ms/consulta)
TF-IDF	CPU (NumPy)	35.1
Embeddings	CPU	42.0
Embeddings	GPU (CUDA)	14.9

En la tabla se puede observar como el metodo de TF-IDF tiene en promedio un tiempo de 35.1ms de consulta, mientras que Embeddings en CPU alcanzo un 42.0, esto permite ver un 19.7% de diferencia a favor de TF-IDF. Cuando Embeddings se ejecuta en GPU mediante CUDA, el tiempo disminuye a 14.9ms por consulta, lo que deja ver que existe una reduccion aproximada del 64.5% respecto a su ejecucion en CPU.

Estos resultados permiten concluir que en general, el metodo TF-IDF presenta un menor tiempo de respuesta cuando ambos metodos se ejecutan usando CPU, sin embargo, Embeddings reduce su tiempo de procesamiento cuando se usa GPU, lo que deja ver que su rendimiento esta condicionado por los recursos del hardware disponible.

D. Umbrales de similitud

Para analizar la capacidad de los metodos de recuperacion de identificar consultas que no pertenecen al contexto de las peliculas se probaron nueve valores entre 0.05 y 0.60, en los que se evaluaron tres grupos: consultas fuera de tema, consultas manuales en español (el conjunto c) y consultas basadas en esloganes (conjunto A). Los resultados muestran que:

En donde se ve que los resultados muestran diferencias entre ambos metodos, en Embeddings, los umbrales que son bajos (0.05 y 0.20) permiten responder todas las preguntas validas de los conjuntos A y C, pero tambien dejan pasar las consultas que estan fuera de tema. Cuando se aumento el umbral a 0.40, las consultas fuera de tema que reciben respuesta disminuyen al 20% mientras que se conservan todas las consultas del conjunto C y el 82% de las consultas del conjunto A. Cuando se tiene un umbral de 0.45 el porcentaje de consultas fuera del tema que son contestadas llega a 0%, por lo que se mantiene el 100% de las consultas de C y el 62% de las consultas de A.

TABLE V
RESULTADOS DEL BARRIDO DE UMBRALES DE SIMILITUD.

Método	Umbral	Fuera de tema que aún responde	Consultas C que responde	Consultas A que responde
Embeddings	0.05	1.00	1.00	1.00
Embeddings	0.10	1.00	1.00	1.00
Embeddings	0.20	1.00	1.00	1.00
Embeddings	0.30	0.70	1.00	0.98
Embeddings	0.40	0.20	1.00	0.82
Embeddings	0.45	0.00	1.00	0.62
Embeddings	0.50	0.00	0.92	0.43
Embeddings	0.55	0.00	0.77	0.24
Embeddings	0.60	0.00	0.51	0.12
TF-IDF	0.05	0.30	0.67	0.99
TF-IDF	0.10	0.30	0.67	0.99
TF-IDF	0.20	0.20	0.36	0.66
TF-IDF	0.30	0.10	0.21	0.14
TF-IDF	0.40	0.10	0.05	0.02
TF-IDF	0.45	0.00	0.00	0.01
TF-IDF	0.50	0.00	0.00	0.01
TF-IDF	0.55	0.00	0.00	0.00
TF-IDF	0.60	0.00	0.00	0.00

TF-IDF por otro lado, con un umbral de 0.20, el 20% de las consultas que estaban fuera del tema aun obtenian respuesta, pero al mismo tiempo, solo el 36% de las consultas de C y el 66% de las consultas de A superan ese valor, es a partir de 0.45 que ninguna consulta fuera de tema supera el umbral, aunque tampoco se conservan consultas validas.

En general, los resultados muestran que Embedding ofrece un comportamiento mas favorable para poder establecer un umbral de aceptación en este experimento, especialmente al considerar las consultas manuales en español. Un umbral de 0.45 permite que el sistema no responda consultas fuera de tema, mientras que se conservan todas las consultas del conjunto C, esta configuracion reduce al 62% la proporcion de las consultas del conjunto A que superan el umbral, por lo que su eleccion implica que haya un equilibrio entre el rechazo a las consultas no pertinentes y a la conservacion de las validas.

E. Capa con LLM

Se realizo una prueba de la capa de LLM con el fin de evaluar su capacidad de recibir una consulta, utilizar el metodo de Embeddings para recuperar las tres peliculas con mayor similitud semantica y generar respuestas redactadas en lenguaje natural a partir usar como contexto los resultados obtenidos mediante embeddings. Para esto, se construye un contexto con los titulos, años, generos y los primeros 300 caracteres de las sinopsis de las peliculas, toda esta informacion se le pasa al LLM, que es el que se encarga de interpretar las opciones y generar una respuesta mas completa relacionada con lo que se le pregunto.

Para comprobar dicho funcionamiento, se realizaron cuatro pruebas con descripciones de peliculas en español:

TABLE VI
RESULTADOS DE LA EVALUACIÓN CUALITATIVA DEL LLM.

Consulta	Película recomendada	Observación
Un barco gigante choca con un iceberg y una chica rica se enamora de un chico pobre.	Show Boat	La recomendación no corresponde a la película esperada (Titanic).
Un robot solitario limpia la Tierra abandonada y se enamora de otro robot.	WALL·E (2008)	La recomendación corresponde a la descripción proporcionada y presenta una relación temática adecuada.
Un viejo ata miles de globos a su casa y viaja a Sudamérica con un niño explorador.	The Blue Butterfly	La recomendación no corresponde a la película esperada (Up).
Una película de terror con casas embrujadas.	The Amityville Horror (2005)	La recomendación presenta relación con la temática solicitada de casas embrujadas.

Se observo que el modelo puede generar respuestas coherentes y contextualizadas, por ejemplo, cuando se le pregunto sobre "Un robot solitario limpia la Tierra abandonada y se enamora de otro robot", el modelo genero una respuesta completa y natural que mencionaba la pelicula Wall-E, sin embargo, se encontraron limitaciones cuando tuvo que redactar otras respuestas, ya que hubo otro caso en el que cuando se le consulto sobre "Un barco gigante choca con un iceberg y una chica rica se enamora de un chico pobre", el modelo recomendo la pelicula Show Boat en vez de la pelicula esperada Titanic, lo que indica que la generacion de respuestas no garantiza la identificacion correcta de la pelicula y que el modelo puede realizar interpretaciones inadecuadas de la consulta.

Esto es debido a que LLM no realiza una busqueda independiente en el catalogo, si no que trabaja a partir de las tres peliculas recuperadas por embeddings, no obstante, el modelo se encarga de interpretar las fichas proporcionadas y decidir cuales considera mas adecuada para la consulta, lo que puede influir en la seleccion final y generar explicaciones que no representen correctamente el contenido de las peliculas, es por esa razon, que se puede concluir que la incorporacion del LLM permite presentar recomendaciones medainte respuestas mas naturales y explicativas que una simple lista de titulos, pero esta capa no garantiza una mayor presicion en la seleccion de las peliculas, por lo que su uso principal se encuentra en la generacion de respuestas en lenguaje natural.

F. Umbrales de similitud

Los intervalos de confianza obtenidos permiten complementar la comparación de los métodos mediante la identificación de la incertidumbre asociada a las diferencias de MRR. En el conjunto A, Embeddings presentó una diferencia positiva de 0,009 respecto a TF-IDF. Sin embargo, el intervalo de confianza comprendido entre -0,004 y 0,021 incluye el valor cero, por lo que no se puede establecer una diferencia concluyente entre ambos métodos para este conjunto.

De manera similar, en el conjunto B, TF-IDF obtuvo un MRR superior al de Embeddings, con una diferencia de 0,020. No obstante, el intervalo de confianza de la diferencia entre Embeddings y TF-IDF, comprendido entre -0,043 y 0,002, también incluye el cero. Por tanto, aunque los resultados observados favorecen a TF-IDF, la estimación mediante bootstrap no permite establecer una diferencia concluyente entre los métodos en este escenario.

En contraste, el conjunto C presentó una diferencia de 0,335 a favor de Embeddings. Su intervalo de confianza, comprendido entre 0,216 y 0,465, no incluye el valor cero y se mantiene completamente en valores positivos.

En consecuencia, el análisis mediante bootstrap respalda los resultados obtenidos con Embeddings para las descripciones. Aunque este método presentó valores de MRR superiores en el conjunto A, y TF-IDF obtuvo mejores resultados en el conjunto B, las diferencias observadas en estos dos escenarios no fueron concluyentes según los intervalos estimados, por lo que esto refuerza la importancia de considerar el tipo de consulta al comparar el comportamiento de los métodos de recuperación.

TABLE VII
INTERVALOS DE CONFIANZA DEL 95% PARA LA DIFERENCIA DE MRR ENTRE TF-IDF Y EMBEDDINGS.

Conjunto	MRR TF-IDF	MRR Embeddings	Diferencia	IC 95%
A. Eslogan	0.070	0.079	0.009	[-0.004; 0.021]
B. Palabras clave	0.209	0.189	-0.020	[-0.043; 0.002]
C. Español (manual)	0.048	0.383	0.335	[0.216; 0.465]

G. Interfaz en Telegram

La interfaz de usuario se implemento en Telegram, lo que permite que los usuarios puedan interactuar con el sistema mediante un chat. La interfaz es sencilla y amigable, permitiendo a los usuarios enviar descripciones de películas y recibir recomendaciones de manera rápida y eficiente:

H. Comparación general de los métodos de recuperación

La comparacion general de los metodos de recuperacion TF-IDF y Embeddings se presenta en la Tabla VIII, donde se observa que Embeddings supera a TF-IDF en el conjunto C, mientras que TF-IDF obtiene mejores resultados en el conjunto B. En el conjunto A, ambos metodos presentan un desempeño similar, con una ligera ventaja para Embeddings. En cuanto al tiempo de respuesta, Embeddings es significativamente mas rapido que TF-IDF, especialmente cuando se ejecuta en GPU:

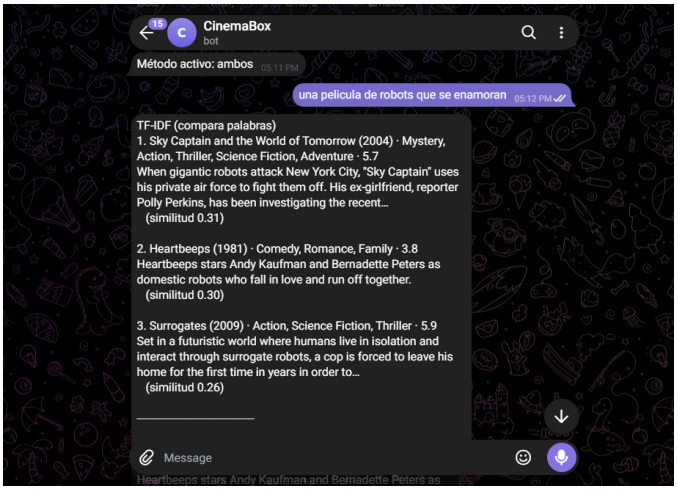


Fig. 2. Interfaz de CinemaBox en Telegram.

TABLE VIII
COMPARACIÓN GENERAL DE LOS MÉTODOS DE RECUPERACIÓN TF-IDF Y EMBEDDINGS.

Conjunto	Método	Top-1	Recall@5	Recall@10	MRR	Tiempo (ms)
A. Eslogan	TF-IDF	0.044	0.089	0.121	0.070	35.1
	Embeddings	0.049	0.107	0.136	0.079	14.9
B. Palabras clave	TF-IDF	0.137	0.285	0.350	0.209	35.1
	Embeddings	0.124	0.242	0.303	0.189	14.9
C. Español (manual)	TF-IDF	0.000	0.128	0.128	0.048	35.1
	Embeddings	0.256	0.513	0.641	0.383	14.9

VIII. DISCUSIÓN

Los resultados obtenidos permiten identificar que el comportamiento de los modelos estan relacionados con las caracteristicas de las consultas que fueron utilizadas durante la evaluacion, los resultados demuestran que no existe un metodo que presente un mejor desempño en todos los conjuntos de datos, ya que la efectividad que se obtenga depende del tipo de informacion proporcionada por el usuario y como esta se relaciona con la informacion de las peliculas en el catalogo.

Por un lado, el modelo Embeddings cuenta con la ventaja de que puede recuperar las peliculas a partir de descripciones en español, mientras que TF-IDF mantiene un comportamiento mas estable cuando las consultas contienen palabras especificas que se encuentren en el catalogo. La incorporacion de LLM aporta una capa de interaccion mas natural que aunque mantiene las limitaciones a la hora de seleccionar e interpretar las peliculas seleccionadas.

IX. CONCLUSIONES

Con este trabajo se desarrollo CinamaBox, un asistente integrado con Telegram que permite a los usuarios realizar consultas y recibir recomendaciones de peliculas mediante dos metodos de recuperacion de informacion: TF-IDF y embeddings multilingues, ambos con la similitud del coseno. de esta manera, se logro cumplir con el objetivo principal del proyecto, que consistia en implementar una solucion que permitiera explorar un catalogo de peliculas mediante consultas realizadas.

Los datos se prepararon a partir de la informacion del dataset TMDB 5000 Movie Dataset, utilizando titulos, generos y sinopsis para representar el contenido de cada pelicula. Por otro lado, el metodo TF-IDF se implemento basandose en la importancia de los terminos y la similitud del coseno, los resultados obtenidos mostraron un mejor desempeño con el conjunto de consultas que incluia palabras clave, mientras que el metodo de embeddings multilingues permirio representar las consultas y las peliculas en un espacio vectorial para comparar su similitud semantica, lo que permitio que presentara un comportamiento mas adecuado a la hora de recuperar peliculas a partir de las consultas en un lenguaje mas natural, como por ejemplo, descripciones de la trama de la pelicula. Por lo tanto, no se identifico un metodo que sea "superior" en todos los casos, sino que hay diferencias en su comportamiento dependieno de sus datos de entrada.

Los tiempos de respuesta fueron en promedio 35.1 ms para el metodo TF-IDF y 42.0 ms para embeddings, sin embargo, si se ejecuta embeddings con GPU, su tiempo promedio se reduce a 14.9 ms, lo que demuestra que el rendimiento del metodo esta condicionado por los recursos del hardware disponible, y que el uso de una GPU puede ayudar a disminuir el tiempo requerido para una respuesta.

Finalmente, la incorporacion de un LLM permitio que el sistema pudiera generar respuestas mas completas y coherentes para el usuario, a partir de la informacion obtenida en la busqueda, lo que mejoro la experiencia del usuario al interactuar con el sistema. En la evaluacion el modelo genero 2 recomendaciones para dos de las cuatro consultas, mientras que para las otras 2 termino seleccionando peliculas que no correspondian con lo esperado, esto permitio identificar que la generacion de respuestas a partir de un modelo de lenguaje no garantiza completamente que la pertinencia de las recomendaciones porque depende de las peliculas recuperadas previamente y de la interpretacion que realiza el modelo sobre la informacion que se le proporciona.

En conclusion, el desarrollo del proyecto permitio implementar y comparar estos dos enfoques de recuperacion de informacion dentro de un asistente conversacional o bot, lo que permitio identificar las fortalezas y debilidades de cada metodo, siendo estas que por un lado TF-IDF tiene la ventaja de mantener la utilidad de las busquedas basandose en palabras clave, mientras que la ventaja de Embeddings es interpretar consultas descriptivas, como desventaja, ambos enfoques presentan limitaciones que permiten establecer oportunidades de mejora para trabajos futuros, especialmente en la ampliación de los conjuntos de evaluación y en el fortalecimiento de los mecanismos de selección y validación de las películas recomendadas.

REFERENCES

[1] DataCamp, "El ciclo de vida del machine learning explicado," https://www.datacamp.com/es/blog/machine-learning-lifecycle-explained, 2023.

[2] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/pdf/2005.11401

[4] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: https://arxiv.org/pdf/2412.15115

[5] S. Robertson, "Understanding inverse document frequency: On theoretical arguments for idf," *Journal of Documentation*, vol. 60, no. 5, pp. 503–520, 2004. [Online]. Available: https://www.researchgate.net/publication/238123710_Understanding_inverse_document_frequency_on_theoretical_arguments_for_IDF

[6] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410.pdf

[7] ——, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. [Online]. Available: https://arxiv.org/pdf/2004.09813

[8] TMDB, "Tmdb 5000 movie dataset," https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata, 2017.